

ETL Data Pipeline Documentation

Olympic Athletes Data Warehouse

Built with Informatica PowerCenter

Technical Design & Implementation Document

1. Project Overview

This document describes the design and implementation of an ETL (Extract, Transform, Load) data pipeline built using Informatica PowerCenter. The pipeline ingests raw Olympic athlete data from flat files, models it into a dimensional (star schema) data warehouse, and orchestrates the load process through a multi-layer architecture with auditing, error handling, and recovery capabilities.

The pipeline follows a four-layer architecture:

- ODS (Operational Data Store) — raw ingestion of source files with no transformation.
- STG (Staging) — data cleansing, joining, deduplication, and preparation of temporary dimension and fact tables.
- DWH_DIM — generation of final dimension tables with surrogate keys.
- DWH_FACT — generation of the final fact table using surrogate key lookups against each dimension.

2. Source Data

The pipeline consumes two source CSV files:

Source File	Columns
Athletes	ID, Name, Sex, Age, Height, Weight, Team, NOC, Games, Year, Season, City, Sport, Event, Medal
NOC (Region Lookup)	NOC, region, notes

The Athletes file contains one row per athlete per event (with repeated athlete records across multiple Games/Events), and the NOC file maps each National Olympic Committee code to its region.

3. Dimensional Modeling — Star Schema

Before building the ETL pipeline, the source data was modeled into a star schema to support analytical querying. The design centers on a single fact table, Fact | Olympic_Athlete, surrounded by five dimension tables.

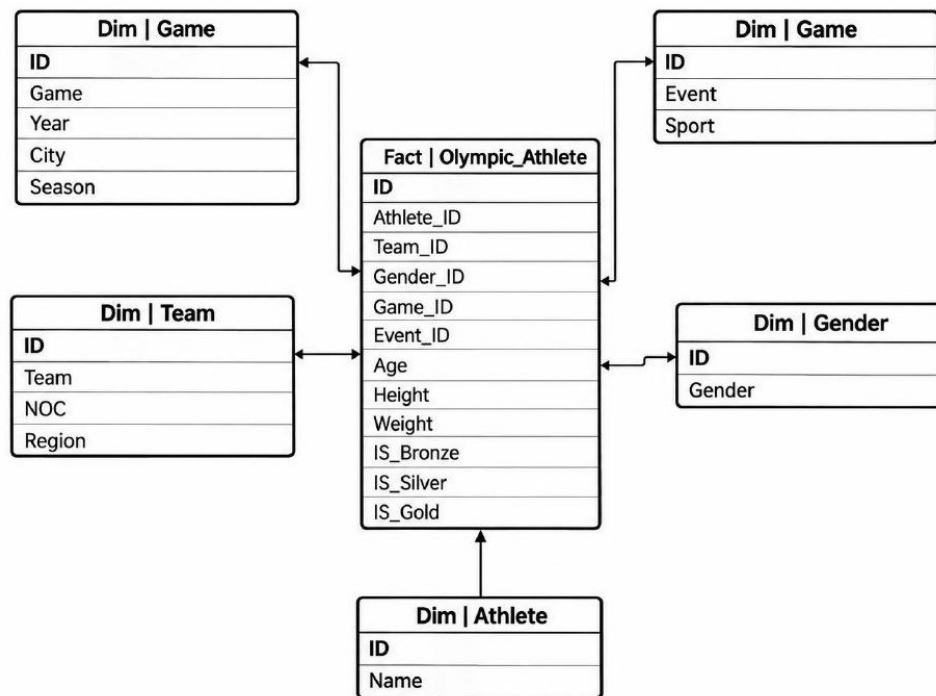


Figure 1 — Star schema: Fact_Olympic_Athlete surrounded by Dim_Game (event/games), Dim_Game (event/sport), Dim_Team, Dim_Gender, and Dim_Athlete.

Table	Type	Key Attributes
Fact Olympic_Athlete	Fact	Athlete_ID, Team_ID, Gender_ID, Game_ID, Event_ID, Age, Height, Weight, IS_Bronze, IS_Silver, IS_Gold
Dim Athlete	Dimension	ID, Name
Dim Team	Dimension	ID, Team, NOC, Region
Dim Gender	Dimension	ID, Gender
Dim Game (Games)	Dimension	ID, Game, Year, City, Season
Dim Game (Event)	Dimension	ID, Event, Sport

A key design decision was to decompose the Medal column into three boolean flag columns on the fact table — IS_Gold, IS_Silver, IS_Bronze — rather than modeling medal as its own dimension, since it behaves as a measure rather than a descriptive attribute.

4. ETL Architecture

The pipeline is organized as a sequential set of layers: Start → ODS → STG / OPS → DWH (DWH_DIM, DWH_FACT). Each stage's success is gated on the status of the previous stage.



Figure 2 — High-level ETL pipeline flow: ODS to Staging (STG) to the Data Warehouse (DWH).

5. ODS Layer — Raw Ingestion

5.1 Design

The ODS layer ingests the two source CSV files directly into two Oracle tables, ATHLETES and NOC, with no business transformation applied. Every column is loaded as a string data type, and an ingestion date column is added to each target table for traceability.

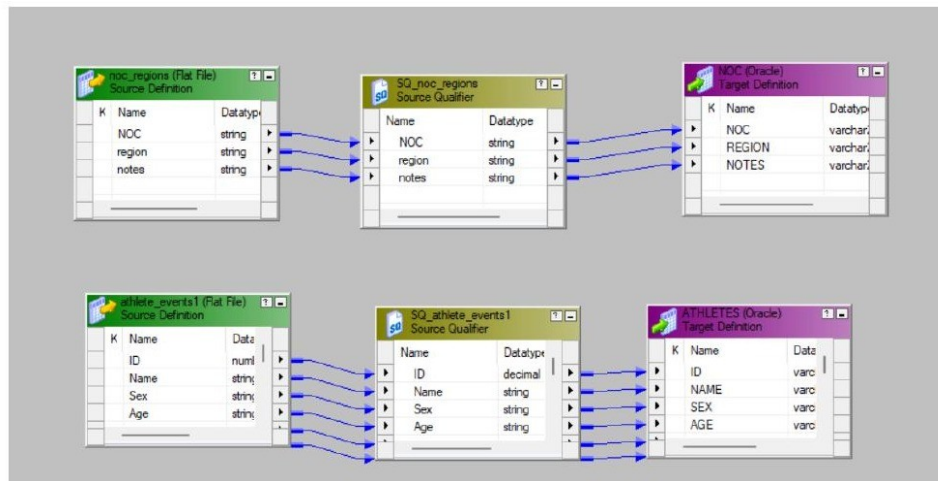


Figure 3 — ODS mappings: noc_regions and athlete_events1 flat files loaded into the NOC and ATHLETES Oracle tables as string columns.

5.2 Issue Encountered — Row Loss Due to Implicit Type Conversion

During initial ODS loads, only 10,370 of the expected 27,000 source rows were loaded. Investigation traced the cause to the Informatica Source Qualifier, which by default maps flat-file columns into typed fields — the Age, Height, and Weight columns were being converted to numeric types. Rows containing the literal value "NA" (used by the source file to represent missing data) failed this conversion, because "NA" is not a null value and cannot be cast to a number.

Resolution: all ODS target columns were kept as string data type end-to-end, so that "NA" values pass through as text rather than being rejected. This preserved every source row for downstream handling.

6. Staging (STG) Layer

6.1 Issue Encountered — Truncated Names in the Reusable Mapplet

A reusable mapplet was created to encapsulate the join logic between the Athletes and NOC data. Its default port data types were set to string with a maximum length of 10 characters. Because the Name column can exceed 10 characters, athlete names were being silently truncated during the transfer.

Name	Datatype	L...
maolet_athlet		
id	string	10
name	string	10
sex	string	10
age	string	10
height	string	10
weight	string	10
team	string	10
noc	string	10
games	string	10
year	string	10
season	string	10
city	string	10
sport	string	10
event	string	10
medal	string	10
OUTPUT		

Figure 4 — Default maplet port lengths, all fixed at string(10), which truncated longer fields such as Name.

Resolution: the maplet's port lengths were widened (to 100 characters) to accommodate the source data without truncation.

6.2 Join Logic — Combining Athletes and NOC

A maplet joins the ATHLETES and NOC tables into a single dataset. NOC is used as the master source (the smaller table) and Athletes as the detail source, with an outer join so that every athlete record is retained even if a matching NOC entry is not found.

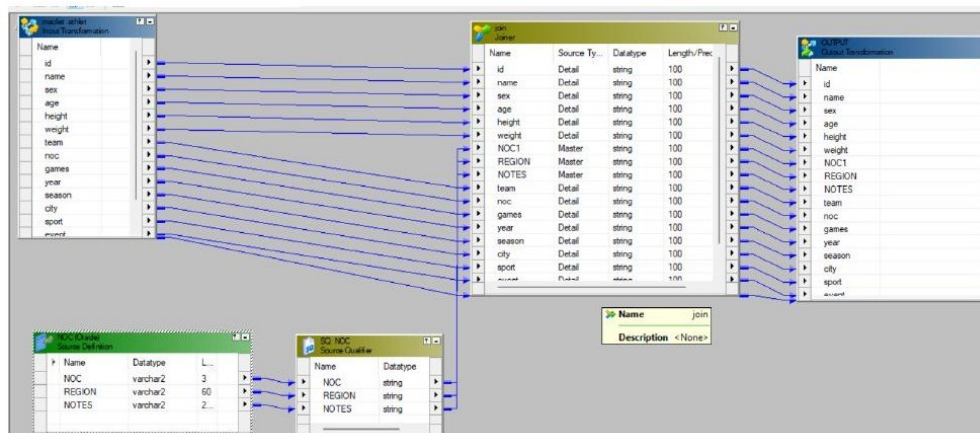


Figure 5 — Reusable maplet joining the NOC and Athlete tables (Master = NOC, Detail = Athlete), with widened 100-character ports.

6.3 Aggregation into Temporary Dimensions and Fact

The maplet's output feeds a set of Aggregator transformations, each producing a deduplicated (grouped) temporary dimension table — DIM_SEX, DIM_EVENT, DIM_OLYMPIQUES, DIM_ATHLETE, DIM_TEAM — plus an Expression transformation that produces a temporary fact table (FACT_OLYMPIQUESEVENT) containing all measures and the natural/business identifiers needed later for dimension lookups.

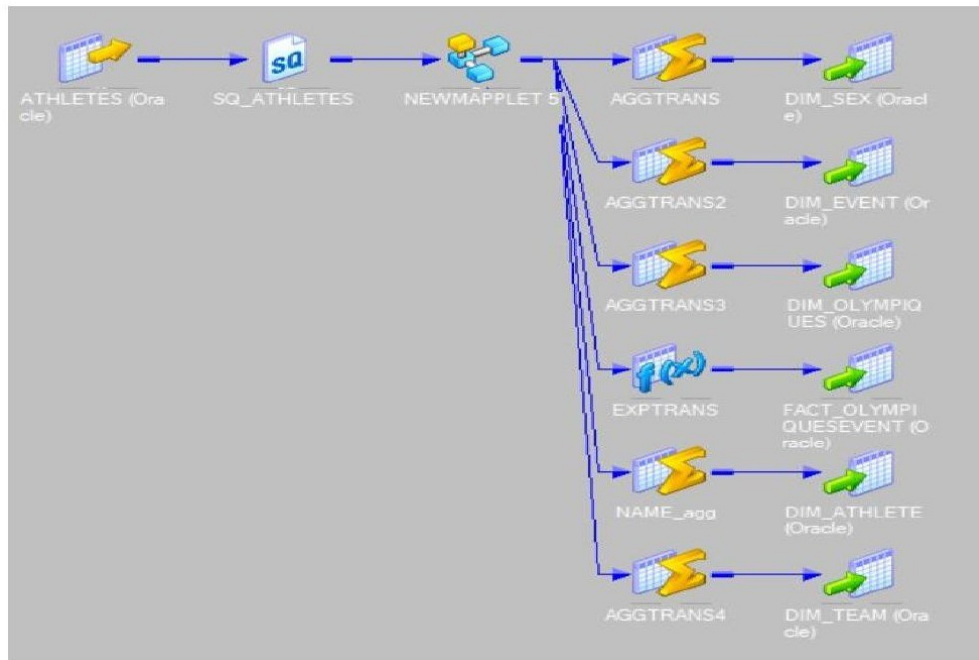


Figure 6 — Staging mappings: the joined mapplet output is grouped into temp dimension tables via Aggregator transformations, and into a temp fact table via an Expression transformation.

A closer view of the same pattern, showing the Aggregator transformations grouping rows into the DIM_SEX and DIM_EVENT temporary tables:

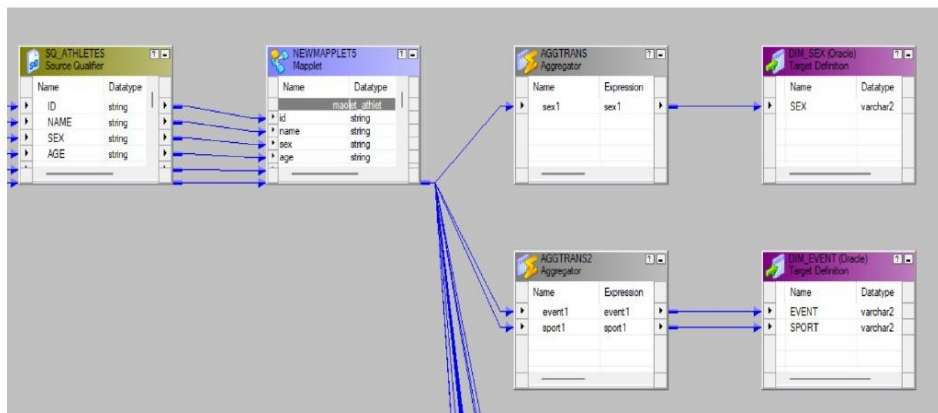


Figure 7 — Detail view: Aggregator transformations (AGGTRANS, AGGTRANS2) grouping athlete rows into DIM_SEX and DIM_EVENT with no duplicates.

6.4 Medal Decomposition

The single Medal source column is converted into three numeric flag columns using conditional (IIF) expressions in the Expression transformation: GOLDMEDAL_FLAG, SILVERMEDAL_FLAG, and BRONZEMEDAL_FLAG. These flags are loaded onto the temporary fact table and, ultimately, the final fact table.

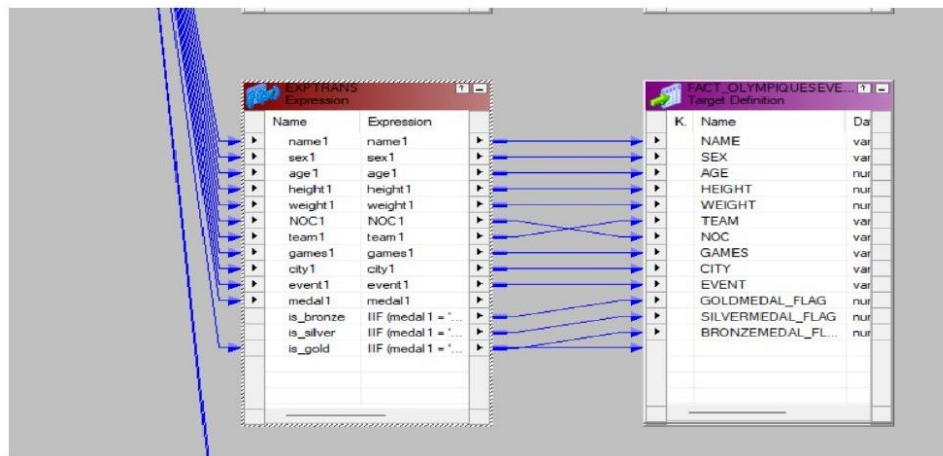


Figure 8 — Expression transformation deriving is_gold / is_silver / is_bronze flags from the Medal column, mapped onto FACT_OLYMPIQUESEVENT.

7. Data Warehouse (DWH) Layer

The DWH layer is split into two mappings, executed in dependency order: dimensions are always loaded before the fact table that references them.

7.1 DWH_DIM — Building Final Dimensions with Surrogate Keys

Each DWH_DIM mapping reads from the corresponding temporary dimension table (populated in STG) and writes to the final dimension table, which uses a surrogate key as its primary key. A Sequence Generator transformation produces this key, incrementing from a configured start value and step for every new row ingested.

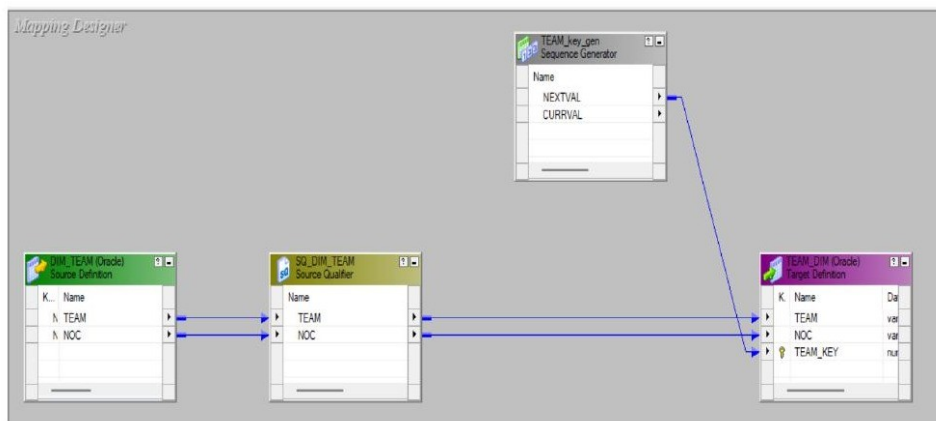


Figure 9 — DWH_DIM mapping example (TEAM_DIM): a Sequence Generator supplies the TEAM_KEY surrogate key for each new team/NOC combination.

7.2 DWH_FACT — Resolving Surrogate Keys via Lookups

The DWH_FACT mapping reads the temporary fact table and, for every dimension, performs a Lookup transformation that matches the temp fact table's business identifier against the corresponding final dimension table. Each lookup returns the dimension's surrogate key, which is written to the final fact table. Measures and medal flags are passed through unchanged from the temp fact table to the final fact table.

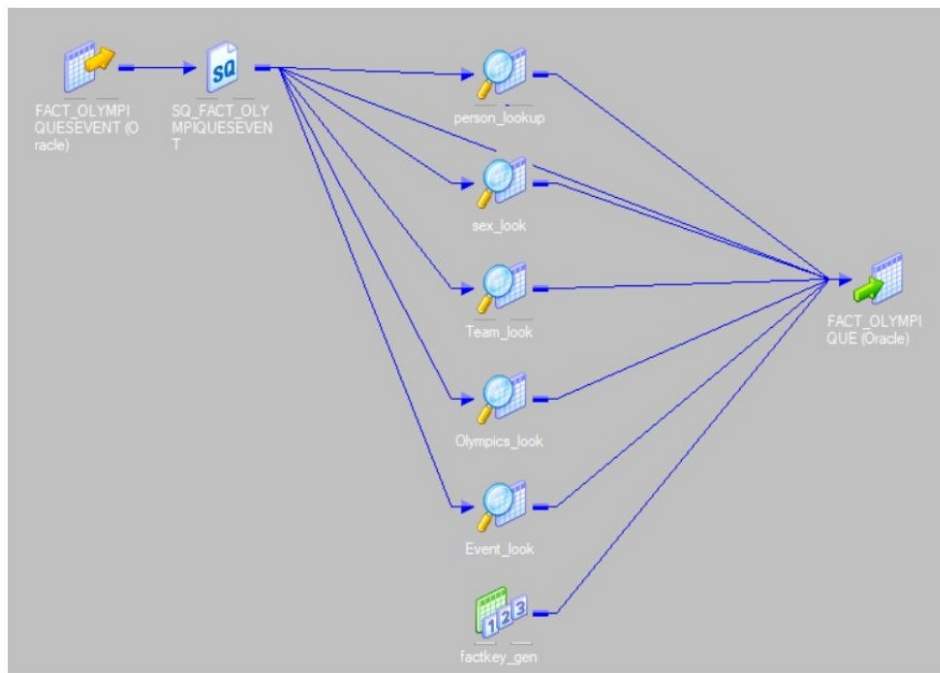


Figure 10 — DWH_FACT mapping: lookups against Person, Sex, Team, Olympics, and Event dimensions resolve each surrogate key before loading FACT_OLYMPIQUE.

8. Workflow Orchestration

8.1 Tasks and Worklets

Each ETL mapping is wrapped in its own session task within the Informatica Workflow Manager: ODS, _STG_, OPS, _DWH_DIM, and DWHFACT, organized inside a folder ("test").

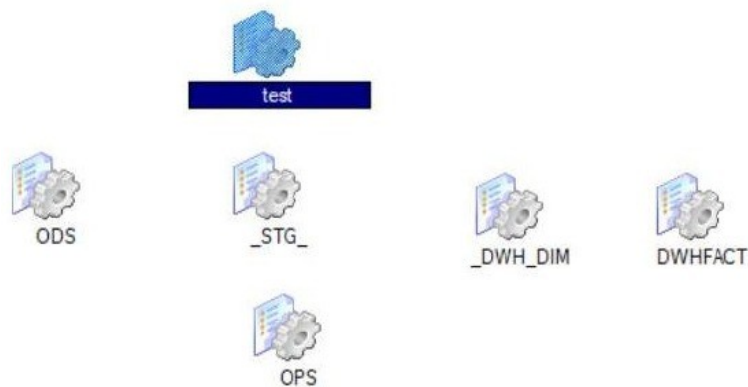


Figure 11 — Workflow Manager session tasks: ODS, _STG_, _DWH_DIM, DWHFACT, and OPS.

A worklet is used to run the STG and OPS sessions in parallel, since they are independent of one another; the worklet also sequences the dimension load ahead of the fact load (DWH_DIM → DWH_FACT).

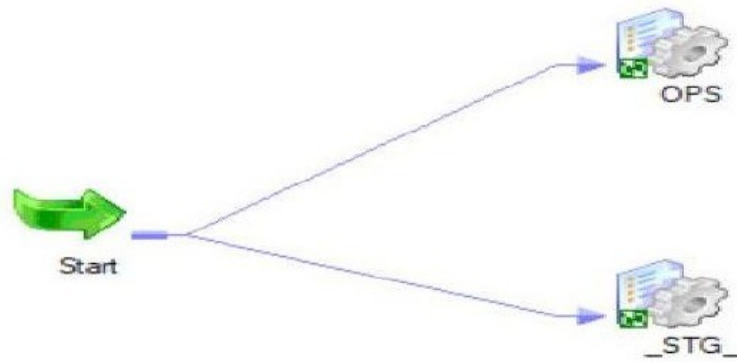


Figure 12 — Worklet: Start branches into OPS and _STG_, which run in parallel.

8.2 Workflow Sequencing

Tasks and worklets are connected in the main workflow and executed end-to-end: Start triggers ODS; on success, STG and OPS run in parallel; on their success, the DWH_DIM and DWHFACT sessions run in sequence.

9. Auditing Framework

An ETL_AUDIT table captures execution metadata for every session in the workflow: AUDIT_ID, WORKFLOW_NAME, SESSION_NAME (task name), START_TIME, END_TIME, source row count, target row count, failed row count, and STATUS.

This is implemented with Pre-SQL and Post-SQL on each session's target transformation:

- Pre-SQL inserts a new audit row when the session starts, using workflow and session variables (e.g. \$PMWorkflowName, \$PMSessionName) and setting STATUS = RUNNING.
- Post-SQL updates that audit row once the session completes, calculating source vs. target row counts, computing failed rows, and setting STATUS to SUCCESS or SUCCESS_WITH_REJECTS.

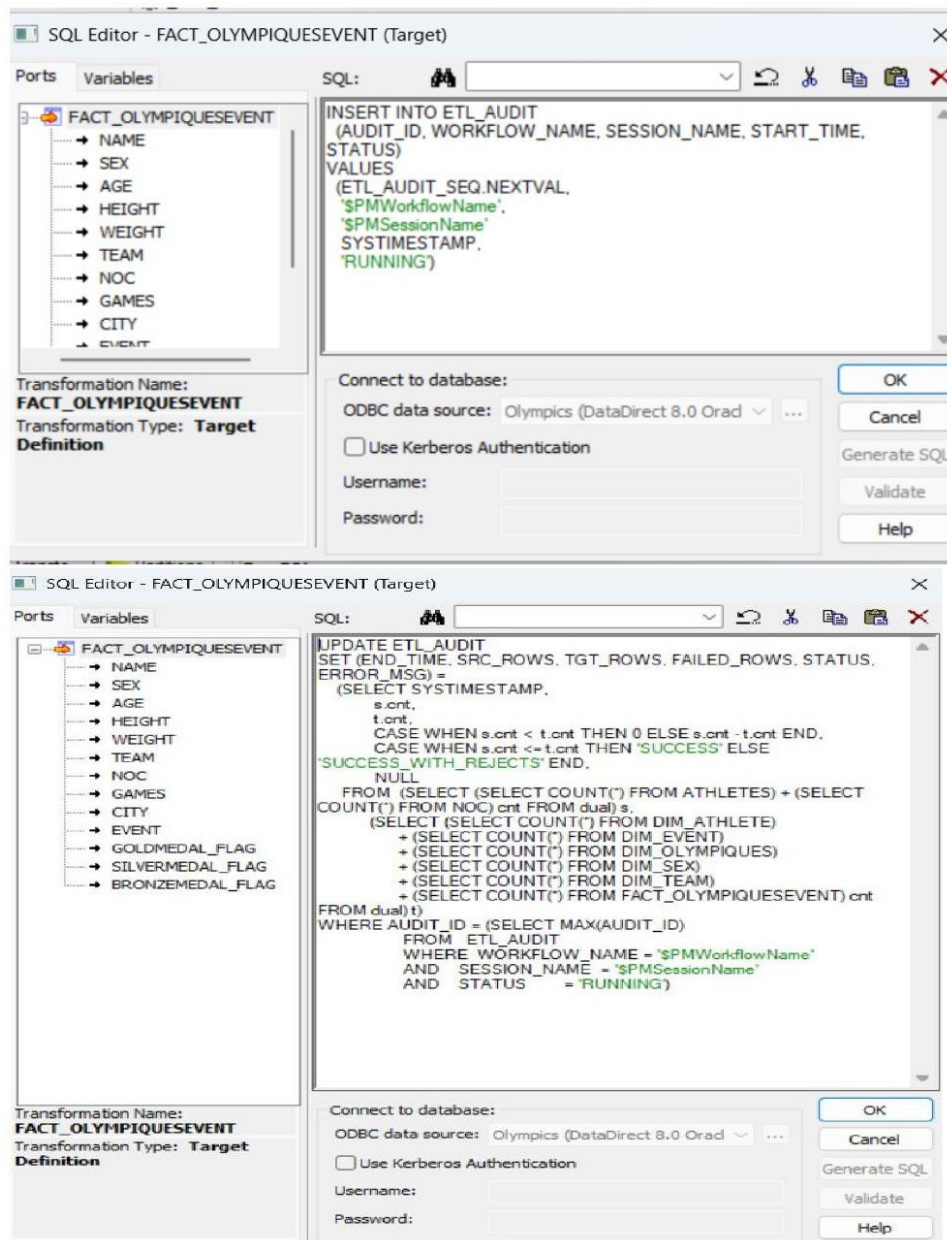


Figure 13 — Pre-SQL (INSERT, sets STATUS = RUNNING) and Post-SQL (UPDATE, computes row counts and final STATUS) on the FACT_OLYMPIQUESEVENT target.

A sample audit run shows the full pipeline executing end-to-end: ODS, OPS, STG, DWH_DIM (which reported 1,211 rejected rows, hence SUCCESS_WITH_REJECTS), and DWHFACT, each completing successfully with matched source and target row counts (aside from expected rejects).

AUDIT_ID	WORKFLOW_NAME	SESSION_NAME	START_TIME	END_TIME	SRC_ROWS	TGT_ROWS	FAILED_ROWS	STATUS
75	139 Olympics_workflow ODS		21-SEP-26 04.50.06.172000000 AM	21-SEP-26 04.50.07.651000000 AM	271346	271346	0	SUCCESS
76	140 Olympics_workflow OPS		21-SEP-26 04.50.10.980000000 AM	21-SEP-26 04.50.12.483000000 AM	271346	271116	0	SUCCESS
77	141 Olympics_workflow STG		21-SEP-26 04.50.16.895000000 AM	21-SEP-26 04.50.19.375000000 AM	271346	407874	0	SUCCESS
78	142 Olympics_workflow DWH_DIM		21-SEP-26 04.50.23.794000000 AM	21-SEP-26 04.50.24.581000000 AM	136758	135547	1211	SUCCESS_WITH_REJECTS
79	143 Olympics_workflow DWHFACT		21-SEP-26 04.50.28.212000000 AM	21-SEP-26 04.50.31.577000000 AM	271116	271116	0	SUCCESS

Figure 14 — Sample ETL_AUDIT log for a full workflow run, showing per-task row counts and status.

10. Recovery Strategy

The pipeline uses a full-load pattern. To avoid reprocessing already-succeeded work after a failure, each session's Recovery Strategy is set to "Resume from last checkpoint," which allows the Workflow Monitor to resume a failed task from its last successful checkpoint rather than reloading rows that already completed.

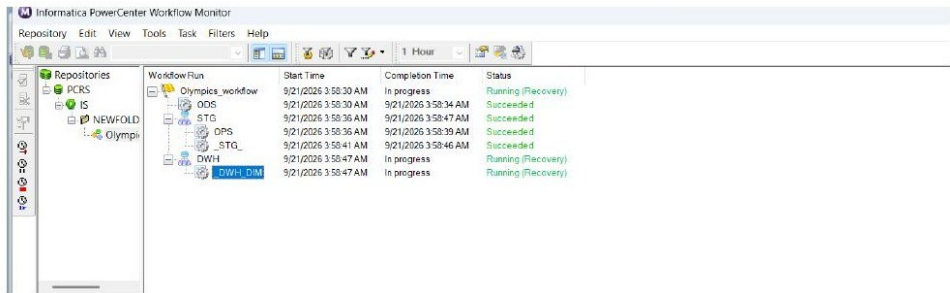


Figure 15 — Workflow Monitor showing a workflow resuming in recovery mode: completed sessions (ODS, STG, OPS) stay Succeeded while DWH and _DWH_DIM show Running (Recovery).

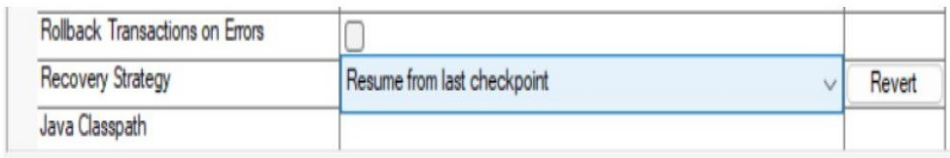


Figure 16 — Session property setting the Recovery Strategy to "Resume from last checkpoint."

11. Error Handling and Notifications

Failures are propagated between tasks using link conditions that check each session's status (e.g. \$ODS.Status = SUCCEEDED) before allowing the next task to run.

18.1- handel failuer between tasks through make status cheak

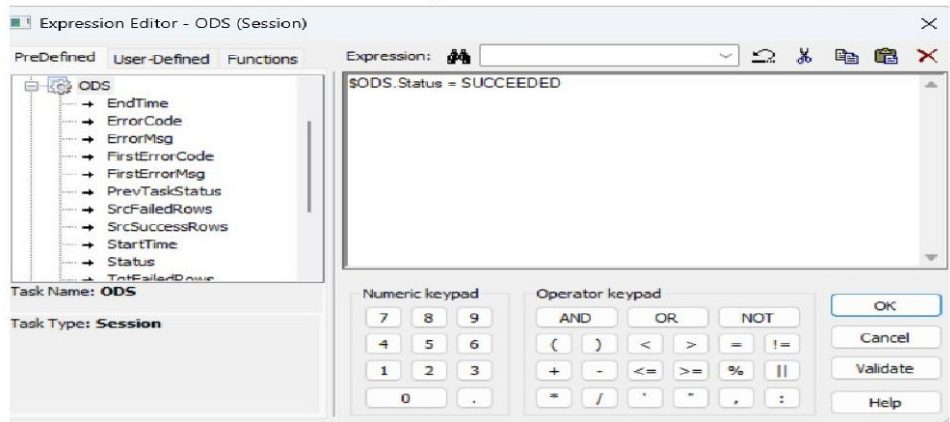


Figure 17 — Link condition expression gating the next task on \$ODS.Status = SUCCEEDED.

When a task fails, an Email task is triggered on the failure path, notifying the responsible engineer with the name of the failed task, its location in the workflow, and a description of the failure.



Figure 18 — Email tasks attached to the failure branch of the ODS and STG sessions.

12. Summary of Key Design Decisions

Area	Decision / Issue	Resolution
ODS data types	Numeric Source Qualifier conversion dropped ~17,000 rows containing "NA"	Load all ODS columns as string; convert types later in STG
Mapplet ports	Default 10-character string ports truncated athlete names	Widened mapplet port lengths to fit source data
Join strategy	Joining Athletes (large) and NOC (small)	NOC as master, Athletes as detail, outer join to retain all athletes
Medal modeling	Medal as a single categorical column	Decomposed into IS_Gold / IS_Silver / IS_Bronze flags on the fact table
Surrogate keys	Need stable warehouse keys independent of source IDs	Sequence Generator transformations per dimension
Fact key resolution	Fact rows must reference dimension surrogate keys	Lookup transformations per dimension in the DWH_FACT mapping
Orchestration	STG and OPS are independent	Run in parallel via a worklet; DWH_DIM always precedes DWHFACT
Observability	Need visibility into row counts and failures per run	Pre/Post-SQL driven ETL_AUDIT table per session
Recovery	Full loads should not restart from zero on failure	"Resume from last checkpoint" recovery strategy
Alerting	Engineers need to know when/where a task fails	Conditional links + Email tasks on failure branches